Universidad de Chile
Facultad de Ciencias Físicas y Matemáticas
Departamento de Ingeniería Eléctrica

Tarea N°2

Clasificador Support Vector Machine

Integrantes: Diego Vega
Profesor: Javier Ruiz del Solar
Auxiliar: Patricio Loncomilla
Ayudantes: Benjamín Llancao Mella
Bruno Cavieres L.
Gonzalo Olguín M.

Fecha de realización: 6 de mayo de 2024
Fecha de entrega: 6 de mayo de 2024
Santiago de Chile

Introducción

En el presente informe se detallara el desarrollo de la tarea Nº2 del curso Inteligencia Computacional. El desarrollo de la tarea se hizo acorde a las indicaciones del documento *Tarea 2 EL4106 2024 wine v04.pdf*.
En el documento mencionado anteriormente se tiene por objetivo el estudio de tres clasificadores pertenecientes a la categoría de los Support Vector Machines. El conjunto de datos utilizados durante la tarea sera *Wine Quality Data Set*, el cual fue entregado por el cuerpo docente en el archivo *winequality-all.csv*. El Data Set proviene del *UC Irvine Machin Learning Repository* y posee 12 columnas, las primeras 11 son las características o **features**, mientras que la última corresponde a la calidad del vino, valor que se encuentra en el intervalo: $[0, 10]$.
Además, es de suma relevancia mencionar que los **kernels** a utilizar en las SVM son:

Lineal $K(x_i, x_j) = x_i^T x_j$

Polinomial $K(x_i, x_j) = (\gamma x_i^T x_j + r)^d$

Gaussiano $K(x_i, x_j) = \exp\left(-\gamma \|x_i^T x_j\|^2\right)$

Por ultimo, los objetivos de la tarea, y por tanto lo que se ha realizado, consisten en:

Manejo de Datos Esta sección consiste en: La lectura del Data Set y la modificación del mismo. División del Data Set en tres conjuntos representativos: el de entrenamiento (60 %), el de validación (20 %) y el de prueba (20 %), verificando la representatividad mediante el calculo de la proporción de cada clase en los conjuntos y asegurándose de que esta sea igual a la del conjunto original. Por ultimo se normalizaran los subconjuntos obtenidos previamente mediante *MinMaxScaler* de *scikit-learn*.

Implementación de un clasificador SVM con kernel Lineal Esta sección consiste en el entrenamiento de un SVM con kernel Lineal, el cual tiene por objetivo discriminar las clases del Data Set. Se implementara mediante *SVC* de *scikit-learn*, para la busqueda de los hiper-parámetros, se utilizara *grid search*. En la grilla se utilizaran 5 valores de C. Además se obtendrá el tiempo de entrenamiento.
Luego, se evaluara el mejor clasificador obtenido, mediante métricas obtenidas sobre el conjunto de validación. Las métricas en cuestión son: Matriz de confusión (en versión normalizada y no normalizada), Curva ROC, AUC, Curva Precision-Recall y Average Precision.

Implementación de un clasificador SVM con kernel Polinomial Esta sección consiste en: La realización de lo mismo que para el clasificador SVM con kernel Lineal con la diferencia que en este caso el kernel sera Polinomial, que por tanto posee otros hiperparámetros.

Implementación de un clasificador SVM con kernel Gaussiano Esta sección consiste en: La realización de lo mismo que para el clasificador SVM con kernel Lineal con

la diferencia que en este el kernel sera Gaussiano, que por tanto posee otros hiperparámetros.

Evaluación y comparación de los clasificadores Esta sección consiste en: La evaluación de los mejores clasificadores obtenidos previamente, utilizando el conjunto de prueba.

Análisis del rendimiento de los clasificadores Esta sección consiste en: El análisis del rendimiento de los mejores clasificadores entrenados, para lo cual se consideraran las métricas obtenidas previamente y el tiempo de entrenamiento.

Índice de Contenidos

1. Desarrollo **1**
1.1. Lectura del Data Set y modificación de columnas 1
1.2. Dividir DataFrame en conjuntos de entrenamiento, validación y prueba . . 2
1.3. Normalizar las características de todos los conjuntos 3
1.4. Support Vector Machine lineal 3
1.4.1. Entrenamiento del clasificador 3
1.4.2. Evaluación del mejor clasificador 4
1.5. Suppport Vector Machine Polinomial 7
1.5.1. Entrenamiento del clasificador 7
1.5.2. Evaluación del mejor clasificador 8
1.6. Support Vector Machine RBF 11
1.6.1. Entrenamiento del clasificador 11
1.6.2. Evaluación del mejor clasificador 12
1.7. Evaluación y comparación de los mejores clasificadores obtenidos 15
1.8. Análisis del rendimiento de los mejores clasificadores obtenidos 18

2. Conclusiones **21**
2.1. Aprendizajes 21
2.2. Dificultades 22
2.3. Evaluación de resultados 23

Anexo A. Código **23**

Índice de Figuras

1. Matriz de confusión del mejor clasificador SVM lineal. 5
2. Matriz de confusión normalizada del mejor clasificador SVM lineal. 6
3. Curva ROC y valor AUC del mejor clasificador SVM lineal. 6
4. Curva Precision-Recall y valor APS del mejor clasificador SVM lineal. . . . 7
5. Matriz de confusión del mejor clasificador SVM polinomial. 9
6. Matriz de confusión normalizada del mejor clasificador SVM polinomial. . . 10
7. Curva ROC y valor AUC del mejor clasificador SVM polinomial. 10
8. Curva Precision-Recall y valor APS del mejor clasificador SVM polinomial. . 11
9. Matriz de confusión del mejor clasificador SVM gaussiano. 13
10. Matriz de confusión normalizada del mejor clasificador SVM gaussiano. . . . 13
11. Curva ROC y valor AUC del mejor clasificador SVM gaussiano. 14
12. Curva Precision-Recall y valor APS del mejor clasificador SVM gaussiano. . . 14
13. Matrices de confusión de los mejores clasificadores. 15
14. Matrices de confusión normalizadas de los mejores clasificadores. 16

15. Curva ROC y valor AUC de los mejores clasificadores. 16
16. Curva Precision-Recall y valor APS de los mejores clasificadores. 17

Índice de Tablas

1. Columnas de los DataFrames. 1
2. Proporciones de las clases en los DataFrames. 2
3. Tiempo de entrenamiento y mejor clasificador SVM Lineal. 4
4. Tiempo de entrenamiento y mejor clasificador SVM Polinomial. 8
5. Tiempo de entrenamiento y mejor clasificador SVM Gaussiano. 12
6. Resultados problema de optimización. 20

Índice de Códigos

1. Carga y modificación del Data Set. 1
2. División del DataFrame. 2
3. Normalización de los subconjuntos de entrenamiento, validación y prueba. . . 3
4. Obtención del mejor clasificador SVM con kernel Lineal. 3
5. Obtención del mejor clasificador SVM con kernel Polinomial. 8
6. Obtención del mejor clasificador SVM con kernel Gaussiano. 12
7. Obtención de las métricas para el problema de optimización. 18
8. Problema de optimización en python. 20
A.1. Jupyter Notebook. 23

1. Desarrollo

1.1. Lectura del Data Set y modificación de columnas

En la presente subsección se realiza la carga del Data Set, para lo cual se utiliza la librería pandas. Además, como el Data Set contenido en el archivo *winequality-all.csv* contiene una columna con al calidad del vino pero en un intervalo $[0, 10]$, se crea una nueva columna llamada *class*, la cual poseerá un valor 1 si qualitiy >= 6 y valor 0 en caso contrario. Lo anterior debido a que los clasificadores SVM utilizados solo permiten la clasificación binaria. El código utilizado para la carga y modificación del Data Set se presenta en 1.

Código 1: Carga y modificación del Data Set.

```
DataSet = 'winequality-all.csv'
DataFrame = pd.read_csv(DataSet, sep=';')
def high_or_low(quality):
  if quality >= 6:
    return 1
  else:
    return 0
DataFrame['class'] = DataFrame['quality'].apply(high_or_low)
DataFrame = DataFrame.drop('quality', axis=1)
```

Lo obtenido mediante la ejecución de 1 se presenta en 1.

Tabla 1: Columnas de los DataFrames.

Columnas del DataFrame de winequality-all.csv
Index(['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', 'pH', 'sulphates', 'alcohol', 'quality'], dtype='object')
Columnas del DataFrame de winequality-all.csv con columna class:
Index(['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', 'pH', 'sulphates', 'alcohol', 'quality', 'class'], dtype='object')
Columnas del DataFrame de winequality-all.csv con columna class y sin quality:
Index(['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', pH', 'sulphates', 'alcohol', 'class'], dtype='object')

Es importante recalcar que el DataFrame que sera utilizado es el tercero de 1, el cual posee la columna *class* pero no la columna *quality*. Por tanto la cantidad de columnas no se modifica con respecto al problema original, manteniendo las dimensionalidades originales

del espacio de características.

1.2. Dividir DataFrame en conjuntos de entrenamiento, validación y prueba

En la presente subsección se divide el DataFrame en tres conjuntos representativos (tres DataFrames representativos), el de entrenamiento, el de validación y el de prueba, con porcentajes del 60 %, 20 % y 20 % respectivamente.
Para realizar la separación se utilizo dos veces la función **train_test_split**, utilizando un **random_state** fijo y *stratify*, para mantener la proporción de clases.
El código utilizado para realizar la división del Data Set se adjunta en 2.

Código 2: División del DataFrame.

```
X = DataFrame.drop('class', axis=1)
y = DataFrame['class']
TrainRatio = 0.6
TestRatio = 0.2
ValidationRatio = 0.2
XTrain, XTemporary, yTrain, yTemporary = train_test_split(X, y, test_size=1-TrainRatio,
    ↪ random_state=3, stratify=y)
XValidation, XTest, yValidation, yTest = train_test_split(XTemporary, yTemporary,
    ↪ test_size=0.5, random_state=3, stratify=yTemporary)
OriginalProportion = [round((y==0).sum()/len(y), 4), round((y==1).sum()/len(y), 4)]
TrainProportion = [round((yTrain==0).sum()/len(yTrain), 4), round((yTrain==1).sum()/
    ↪ len(yTrain), 4)]
ValidationProportion = [round((yValidation==0).sum()/len(yValidation), 4), round((
    ↪ yValidation==1).sum()/len(yValidation), 4)]
TestProportion = [round((yTest==0).sum()/len(yTest), 4), round((yTest==1).sum()/len(
    ↪ yTest), 4)]
```

Las proporciones obtenidas mediante la ejecución de 2 se presentan en 2.

Tabla 2: Proporciones de las clases en los DataFrames.

Proporción de las clases en el DataFrame original.	$[0.3669, 0.6331]$
Proporción de las clases en el DataFrame de entrenamiento.	$[0.3669, 0.6331]$
Proporción de las clases en el DataFrame de validación.	$[0.3672, 0.6328]$
Proporción de las clases en el DataFrame de prueba.	$[0.3669, 0.6331]$

Como se puede ver en 2, la proporción entre las clases del DataFrame original se logran mantener con exactitud en el caso de los subconjuntos de entrenamiento y prueba, mientras que para el subconjunto de validación hay una diferencia de ± 0.0003, la cual no es significativa, por lo que es posible aseverar que se mantienen la proporción del Data Set original en los subconjuntos creados, lo cual es de suma relevancia pues así se mantiene la representatividad

de las muestras del Data Set original.

1.3. Normalizar las características de todos los conjuntos

En la presente subsección se normalizan las características de los 3 subconjuntos creados en 1.2, es decir, se normalizan los subconjuntos de entrenamiento, validación y prueba. Para esto, se utilizo la función **MaxMinScaler** de *scikit-learn*. La función fue entrenada con el conjunto de entrenamiento y posteriormente se utilizo sobre los conjuntos de entrenamiento, validación y prueba, obteniendo así los conjuntos normalizados.
El código utilizado para realizar la normalización de los datos se adjunta en 3.

Código 3: Normalización de los subconjuntos de entrenamiento, validación y prueba.

```
Scaler = MinMaxScaler()
Scaler.fit(XTrain)
XTrainNormalized = Scaler.transform(XTrain)
XValidationNormalized = Scaler.transform(XValidation)
XTestNormalized = Scaler.transform(XTest)
```

Es importante mencionar que durante el resto del desarrollo se utilizaran los DataFrames normalizados.

1.4. Support Vector Machine lineal

1.4.1. Entrenamiento del clasificador

En la presente subsección se creo y entreno un clasificador Support Vector Machine con kernel Lineal. Además, para la elección del clasificador se utilizo una grilla para encontrar el mejor hiper-parámetro, en este caso **C**. Es importante mencionar que **C** corresponde al parámetro de regularización, la cual es responsable de controlar la compensación entre tener una decisión suave y la clasificación correcta de la mayoría de las muestras del entrenamiento.
Para no hacer uso de la validación cruzada, se utilizo la función *PredefindedSplit* de *scikit-learn*, la cual permite definir la división de los datos a utilizar durante la validación cruzada, en este caso particular, se utilizo para señalar que muestras pertenecían al conjunto de entrenamiento y cuales al de validación.
Mientras que para hacer uso de una grilla, se utilizo la función *GridSearchCV* de *scikit-learn*, la cual realiza la búsqueda del mejor hiper-parámetro de una grilla utilizando validación cruzada, aunque como se menciono previamente, se definirán los pliegues mediante *PredefinedSplit*, por lo que se «ignorara» la validación cruzada.
El código utilizado para la creación y entrenamiento del clasificador SVM con kernel Lineal se adjunta en 4.

Código 4: Obtención del mejor clasificador SVM con kernel Lineal.

```
Indexs = np.zeros(len(XTrainNormalized) + len(XValidationNormalized))
Indexs[:len(XTrainNormalized)] = -1
PreDefinedSplit = PredefinedSplit(test_fold=Indexs)
ParametersGrid = {'C': [0.01, 0.1, 1, 10, 100]}
SVML = SVC(kernel='linear')
GridSearch = GridSearchCV(estimator=SVML, param_grid=ParametersGrid, cv=
    ↪ PreDefinedSplit)
InitialTrainingTimeLinear = time.time()
GridSearch.fit(np.vstack((XTrainNormalized, XValidationNormalized)), np.hstack((yTrain,
    ↪ yValidation)))
FinalTrainingTimeLinear = time.time()
BestSVML = GridSearch.best_estimator_
BestSVML.fit(XTrainNormalized, yTrain)
```

Los resultados obtenidos mediante la ejecución de 4 se presentan en 3.

Tabla 3: Tiempo de entrenamiento y mejor clasificador SVM Lineal.

Tiempo de entrenamiento.	16.535385847091675
Mejor clasificador SVM Lineal.	SVC(C=10, kernel=linear)

Como se puede visualizar en 3, la mejor configuración para el SVM Lineal es aquella con el hiper-parámetro $\mathbf{C} = 10$.
Además, el mejor clasificador tarda alrededor de 17 segundos en terminar el entrenamiento.
Es importante señalar que para el caso de la función*PredefinedSplit* se utilizo como parámetro *test_fold* un arreglo de *numpy* que declara cuales valores son del conjunto de entrenamiento (-1) y cuales son del conjunto de validación (0).
En cuanto a la función *GridSearchCV* se utilizo por parámetros a: El clasificador (*estimator*. SVM Lineal), la grilla de parámetros (*param_grid*. Grilla que explora C) y la partición de la no validación cruzada (*cv*. Partición generada por PredefinedSplit).
Finalmente, para el calculo del tiempo de entrenamiento, se utilizo *time* de la librería *time*.

1.4.2. Evaluación del mejor clasificador

En la presente subsección se evalúo el mejor clasificador obtenido en 4, para lo cual se hizo uso de diversas métricas, las que se obtuvieron mediante diversas funciones de la librería *scikit-learn*. A partir de las métricas obtenidas se generaron varias gráficas.
Es importante mencionar que se realizo la predicción sobre el conjunto de validación, con la cual se obtuvo la matriz de confusión. Para el resto de las gráficas se utilizo *decision_function* sobre el conjunto de validación.
Las gráficas generadas se presentan a continuación.

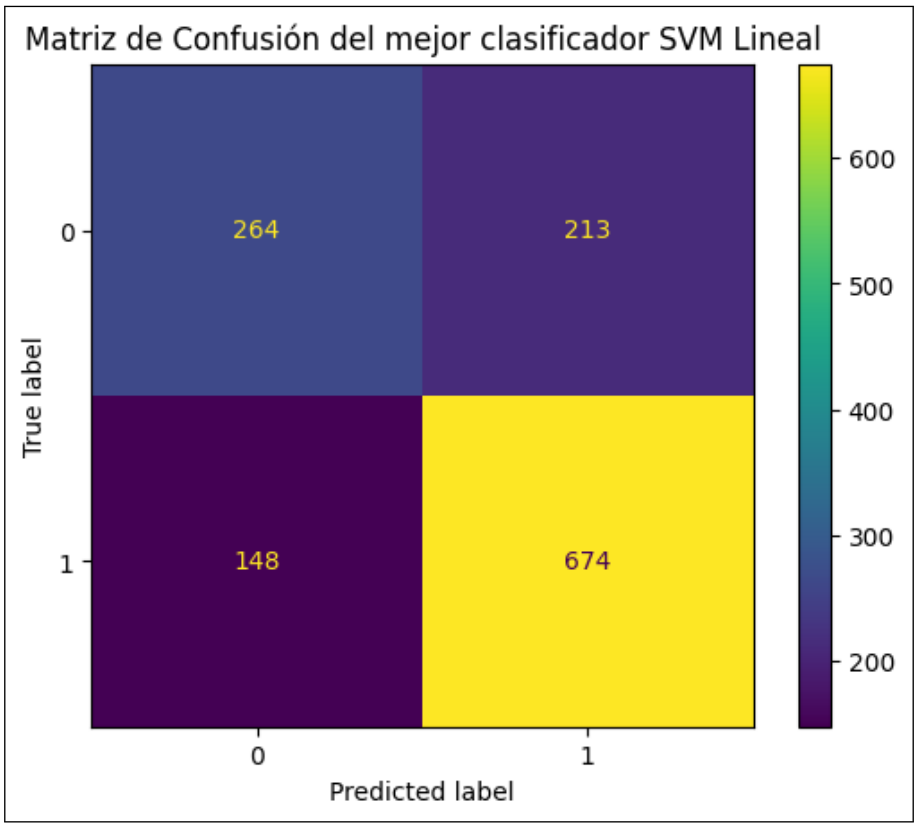


Figura 1: Matriz de confusión del mejor clasificador SVM lineal.

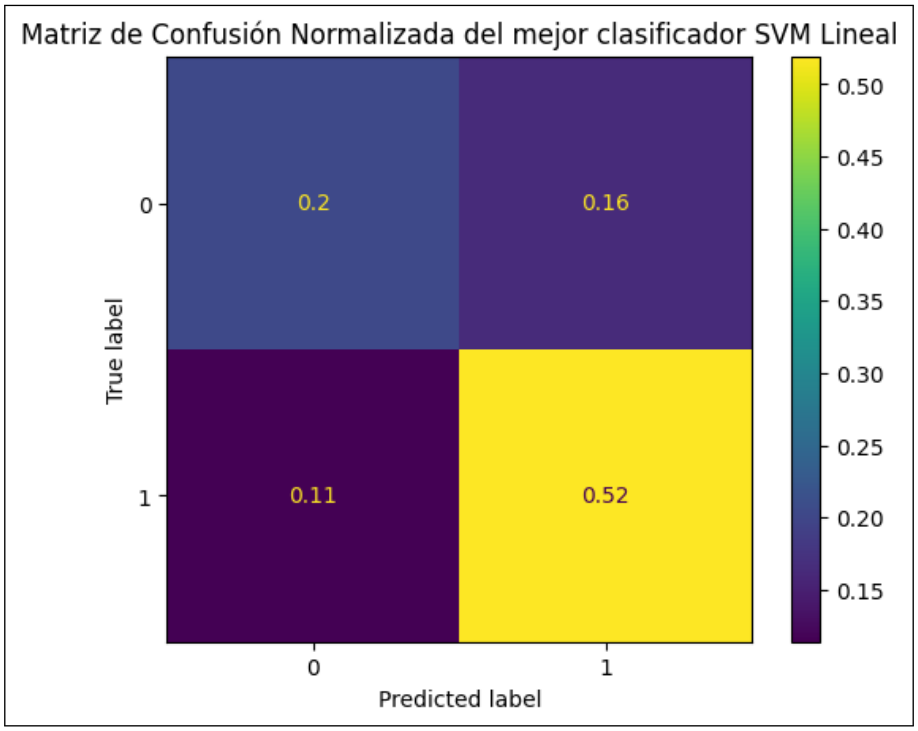


Figura 2: Matriz de confusión normalizada del mejor clasificador SVM lineal.

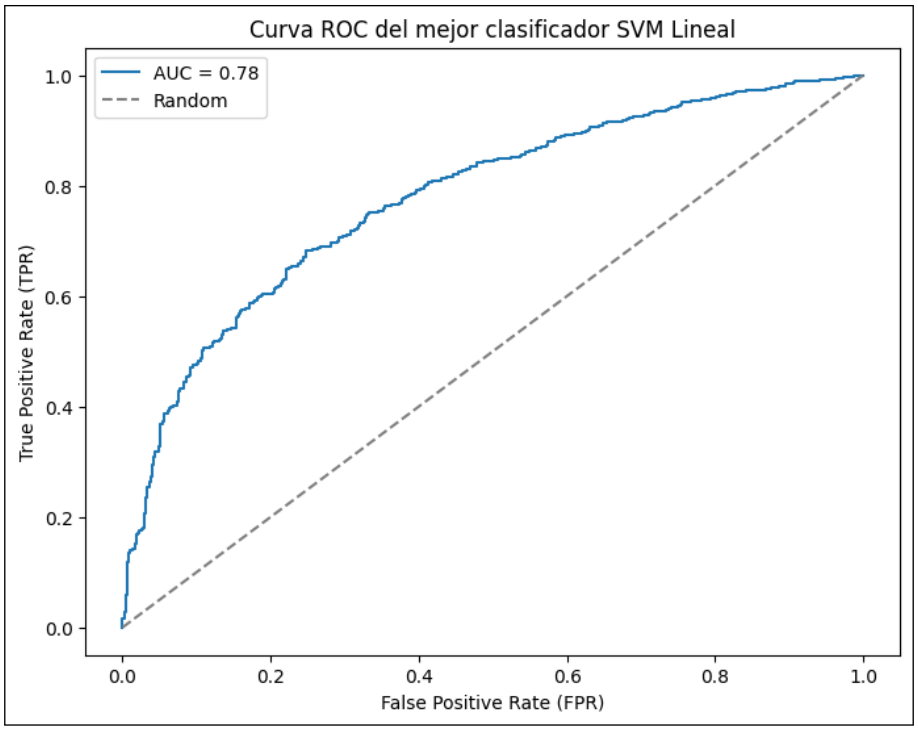


Figura 3: Curva ROC y valor AUC del mejor clasificador SVM lineal.

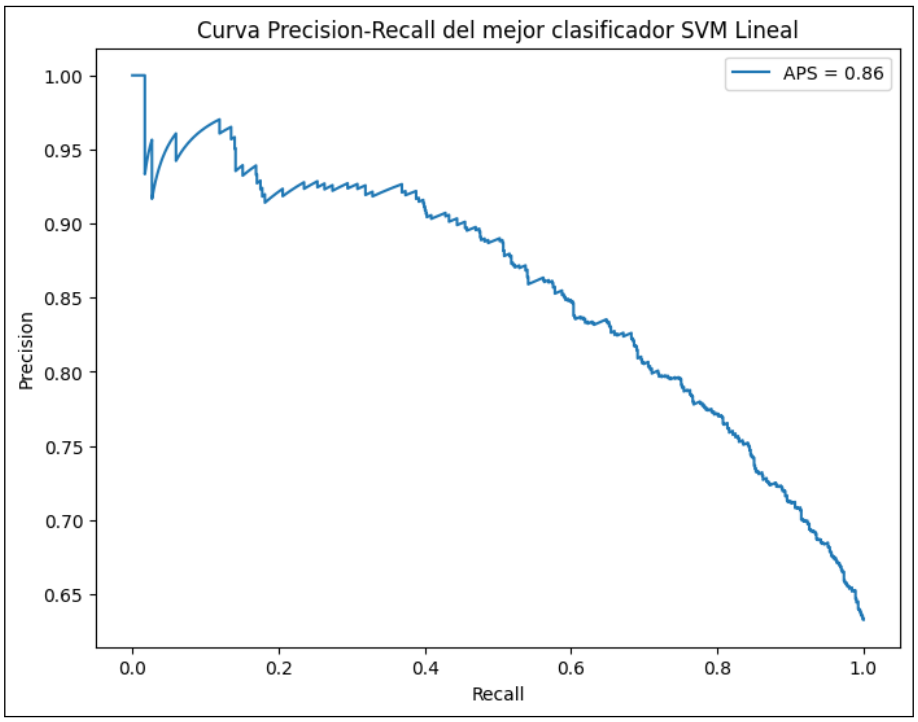


Figura 4: Curva Precision-Recall y valor APS del mejor clasificador SVM lineal.

De la **matriz de confusión** representada en 1 y 2, es posible observar que el valor de TP es superior al resto, incluso siendo mayor a la mitad del total de casos, le sigue el valor TN, luego el valor de FN y por ultimo el de FP, por lo que el clasificador obtenido es «bueno», ya que los primeros dos valores mencionados son aciertos, y suman un 72 %, mientras que el 28 % restante son los errores.
De la curva **ROC** 3 se puede mencionar que es una curva «buena», pues la pendiente inicial es bastante pronunciada, es una curva suave y se aleja bastante de la curva **Random**. Respecto al valor **AUC** obtenido, se puede mencionar que es alto, pero no lo suficientemente como para considerar que el clasificador es muy bueno.
De la curva **Precision-Recall** 4 se puede mencionar que es una curva medianamente «buena», pues la pendiente inicial no es tan pronunciada y no es una curva suave. Respecto al valor **APS** obtenido, se puede mencionar que es alto e indica que el modelo tiene una alta precisión al clasificar los elementos de la clase positiva. Esto confirma lo mencionado en el análisis de la **matriz de confusión**.

1.5. Suppport Vector Machine Polinomial

1.5.1. Entrenamiento del clasificador

En la presente subsección se creo y entreno un clasificador Support Vector Machine con kernel Polinomial. Además, para la elección del clasificador se utilizo una grilla para encontrar los mejores hiper-párametros, en este caso **C** y **degree**. Es importante mencionar que

degree corresponde al grado del polinomio del kernel Polinomial, mientras que **C** se explico en 1.4.1. Por ultimo, se realizara la búsqueda de la grilla sobre 3 valores de **C** y 3 valores de **degree**.
Nuevamente se utilizo *PredefinedSplit* para «ignorar» la validación cruzada de *GridSearchCV*.
El código utilizado para la creación y entrenamiento del clasificador SVM con kernel Polinomial se adjunta en 5.

Código 5: Obtención del mejor clasificador SVM con kernel Polinomial.

```
ParametersGrid = {'C': [0.1, 1, 10], 'degree': [2, 3, 4]}
SVMP = SVC(kernel='poly', coef0=1)
GridSearch = GridSearchCV(estimator=SVMP, param_grid=ParametersGrid, cv=
    ↪ PreDefinedSplit)
InitialTrainingTimePolynomial = time.time()
GridSearch.fit(np.vstack((XTrainNormalized, XValidationNormalized)), np.hstack((yTrain,
    ↪ yValidation)))
FinalTrainingTimePolynomial = time.time()
BestSVMP = GridSearch.best_estimator_
BestSVMP.fit(XTrainNormalized, yTrain)
```

Los resultados mediante la ejecución de 5 se presentan en 4.

Tabla 4: Tiempo de entrenamiento y mejor clasificador SVM Polinomial.

Tiempo de entrenamiento.	114.93021178245544
Mejor clasificador SVM Polinomial.	SVC(C=10, degree=4, coef0=1, kernel=poly)

Como se puede visualizar en 4, la mejor configuración de hiper-parámetros encontrada para el SVM Polinomial es aquella con $\mathbf{C} = 10$, $\mathbf{degree} = 4$ y $\mathbf{coef0} = \mathbf{1}$.
Además, el mejor clasificador tarda alrededor de 115 segundos en terminar el entrenamiento.
El parámetro **coef0** corresponde al r de la función kernel definida en la introducción, es decir, dicho parámetro corresponde al valor independiente definido en la función del kernel y por enunciado se estableció en 1.

1.5.2. Evaluación del mejor clasificador

En la presente subsección se evaluó el mejor clasificador obtenido en 1.5.1, para lo cual se hizo uso de las mismas métricas que en 1.4.2.
Las gráficas generadas se presentan a continuación.

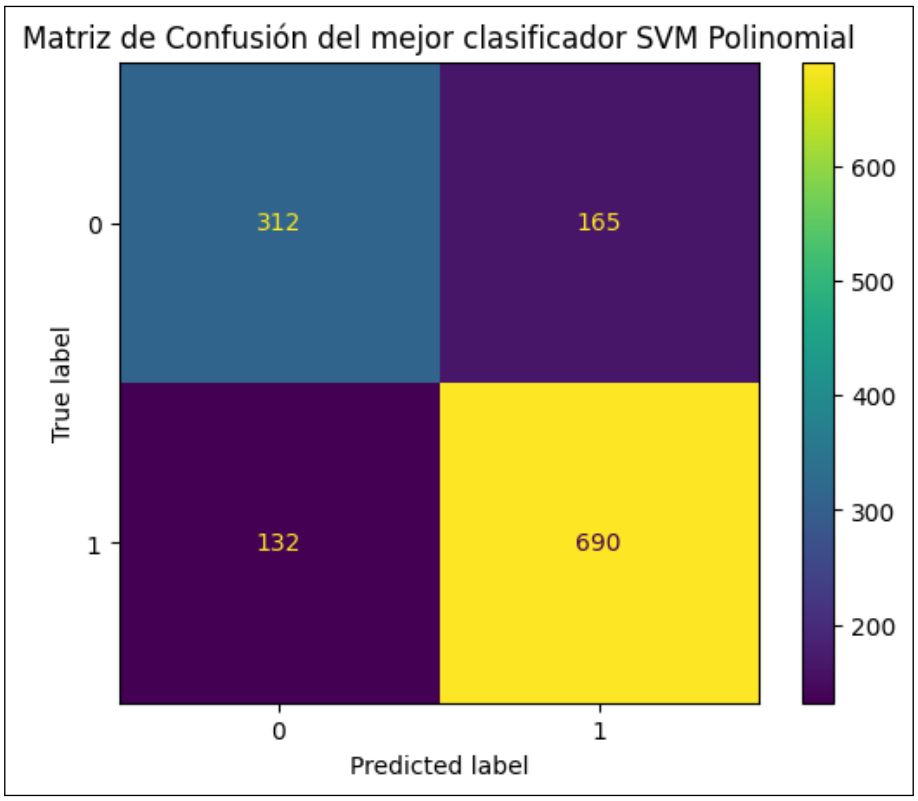


Figura 5: Matriz de confusión del mejor clasificador SVM polinomial.

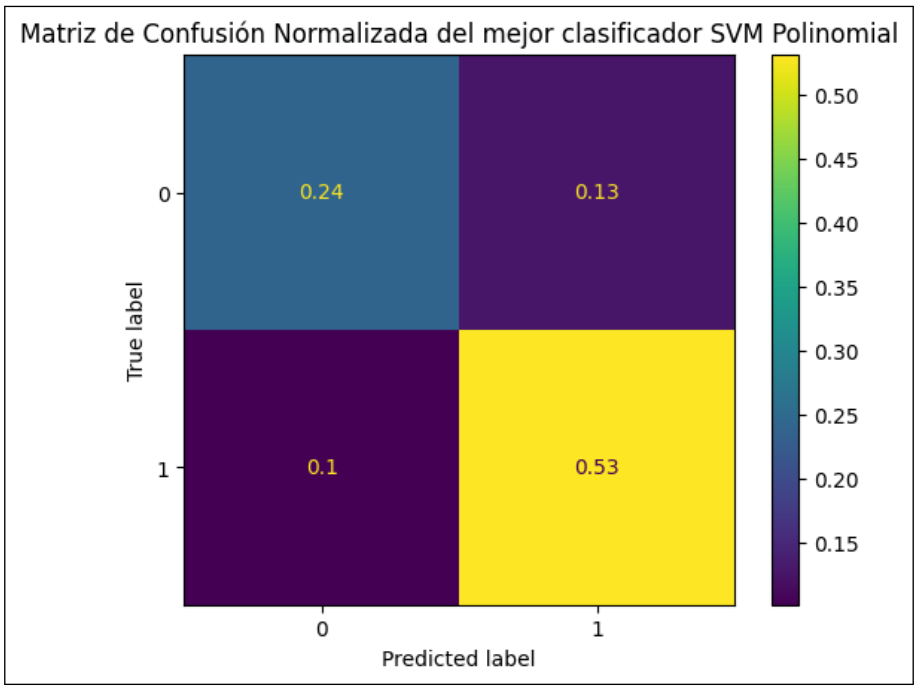


Figura 6: Matriz de confusión normalizada del mejor clasificador SVM polinomial.

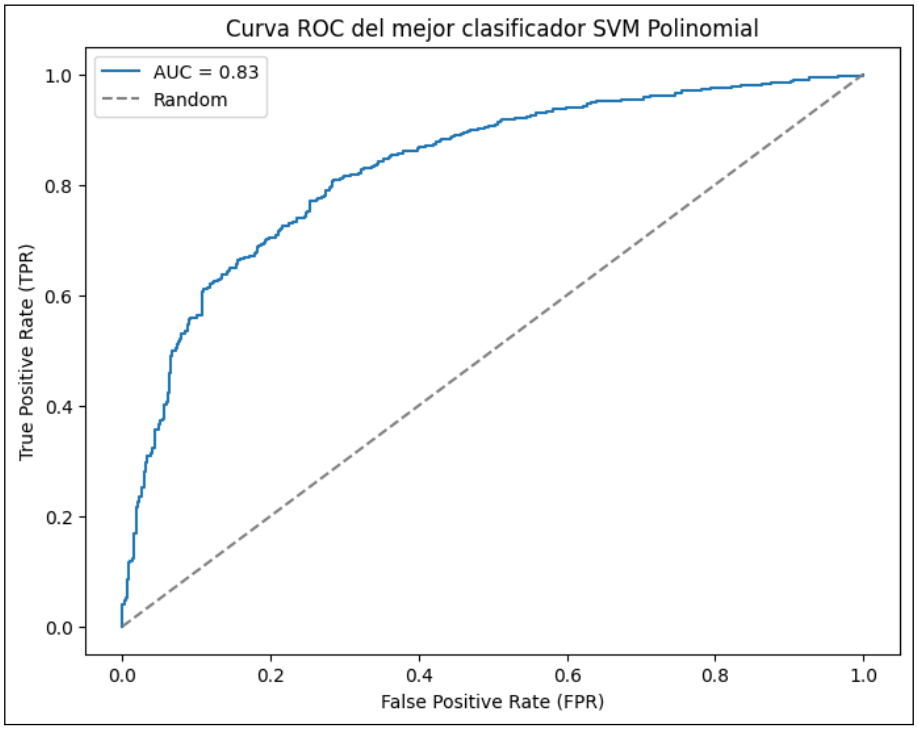


Figura 7: Curva ROC y valor AUC del mejor clasificador SVM polinomial.

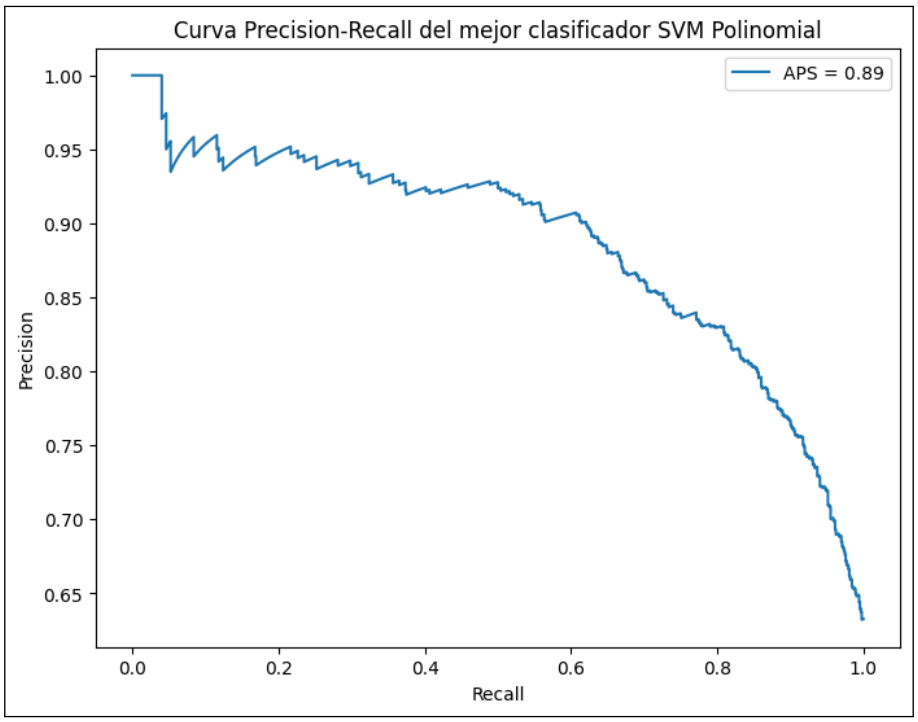


Figura 8: Curva Precision-Recall y valor APS del mejor clasificador SVM polinomial.

De la **matriz de confusión** representada en 5 y 6, es posible observar que el valor de TP es superior al resto, incluso siendo mayor a la mitad del total de casos, le sigue el valor TN, luego el valor de FN y por ultimo el de FP, por lo que el clasificador obtenido es «muy bueno», ya que los primeros dos valores mencionados son aciertos, y suman un 77 %, mientras que el 23 % restante son los errores.
De la curva **ROC** 7 se puede mencionar que es una curva «buena», pues la pendiente inicial es bastante pronunciada, es una curva suave y se aleja bastante de la curva **Random**. Respecto al valor **AUC** obtenido, se puede mencionar que es alto, lo suficientemente como para considerar que el clasificador es muy bueno.
De la curva **Precision-Recall** 8 se puede mencionar que es una curva «buena», pues la pendiente inicial es pronunciada, pero no es una curva suave. Respecto al valor **APS** obtenido, se puede mencionar que es muy alto e indica que el modelo tiene una alta precisión al clasificar los elementos de la clase positiva. Esto confirma lo mencionado en el análisis de la **matriz de confusión**.

1.6. Support Vector Machine RBF

1.6.1. Entrenamiento del clasificador

En la presente subsección se creo y entreno un clasificador Support Vector Machine con kernel Gaussiano. Además, para la elección del clasificador se utilizo una grilla para encontrar los mejores hiper-parámetros, en este caso **C** y **gamma**. Es importante mencionar que **gam-**

ma corresponde al parámetro que controla la influencia de un ejemplo del entrenamiento de una muestra en la clasificación de las otras muestras. Por ultimo, se realizara la búsqueda de la grilla sobre 5 valores de **C** y 4 valores de **gamma**.
El código utilizado para la creación y entrenamiento del clasificador SVM con kernel Gaussiano se adjunta en 6.

Código 6: Obtención del mejor clasificador SVM con kernel Gaussiano.

```
ParametersGrid = {'C': [0.01, 0.1, 1, 10, 100], 'gamma': [0.01, 0.1, 1, 10]}
SVMRBF = SVC(kernel='rbf')
GridSearch = GridSearchCV(estimator=SVMRBF, param_grid=ParametersGrid, cv=
    ↪ PreDefinedSplit)
InitialTrainingTimeRBF = time.time()
GridSearch.fit(np.vstack((XTrainNormalized, XValidationNormalized)), np.hstack((yTrain,
    ↪ yValidation)))
FinalTrainingTimeRBF = time.time()
BestSVMRBF = GridSearch.best_estimator_
BestSVMRBF.fit(XTrainNormalized, yTrain)
```

Los resultados mediante la ejecución de 6 se presentan en 5.

Tabla 5: Tiempo de entrenamiento y mejor clasificador SVM Gaussiano.

Tiempo de entrenamiento.	22.660074949264526
Mejor clasificador SVM Gaussiano.	SVC(C=10, gamma=10, kernel=rbf)

Como se puede visualizar en 5, la mejor configuración de hiper-parámetros encontrada para el SVM Gaussiano es aquella con $\mathbf{C} = 10$ y $\mathbf{gamma} = 10$.
Además, el mejor clasificador tarda alrededor de 23 segundos en terminar el entrenamiento.

1.6.2. Evaluación del mejor clasificador

En la presente subsección se evaluó el mejor clasificador obtenido en 1.6.1, para lo cual se hizo uso de las mismas métricas que en 1.4.2.
Las gráficas generadas se presentan a continuación.

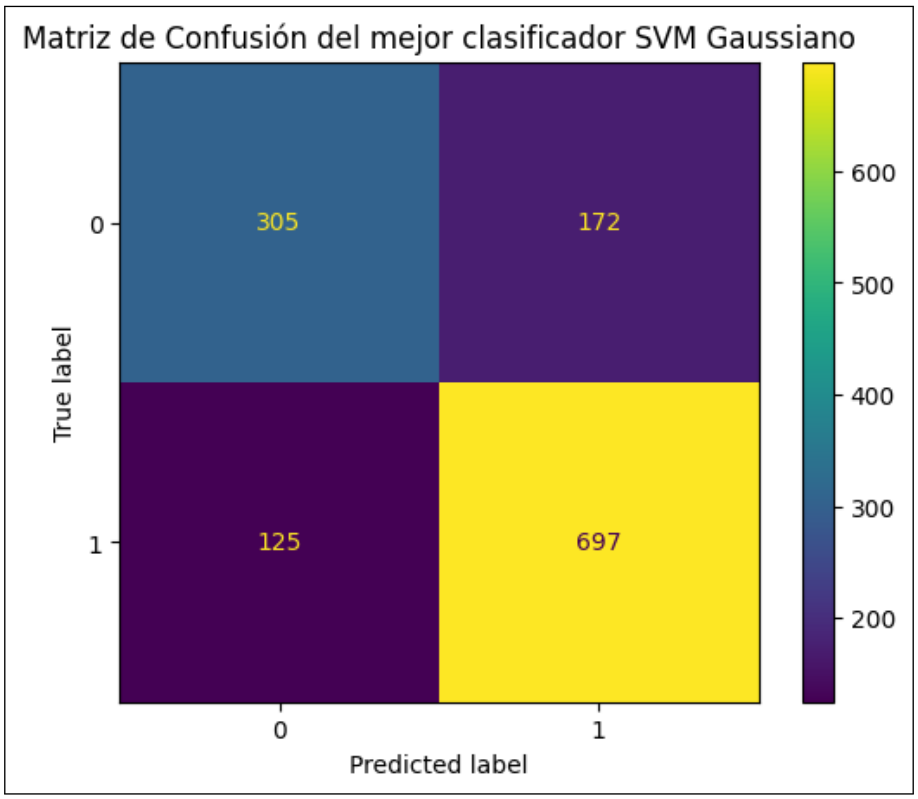


Figura 9: Matriz de confusión del mejor clasificador SVM gaussiano.

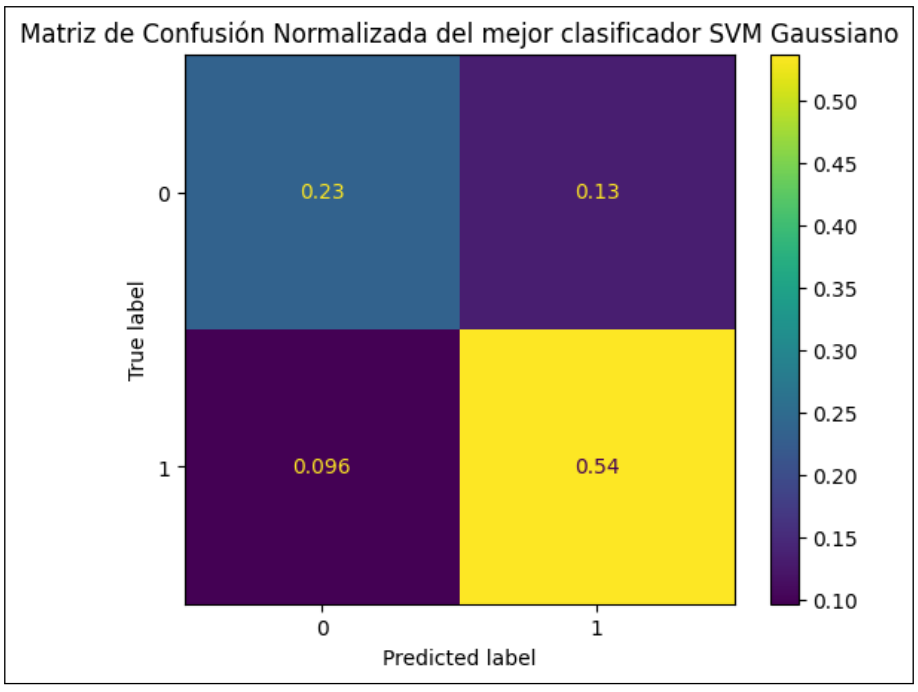


Figura 10: Matriz de confusión normalizada del mejor clasificador SVM gaussiano.

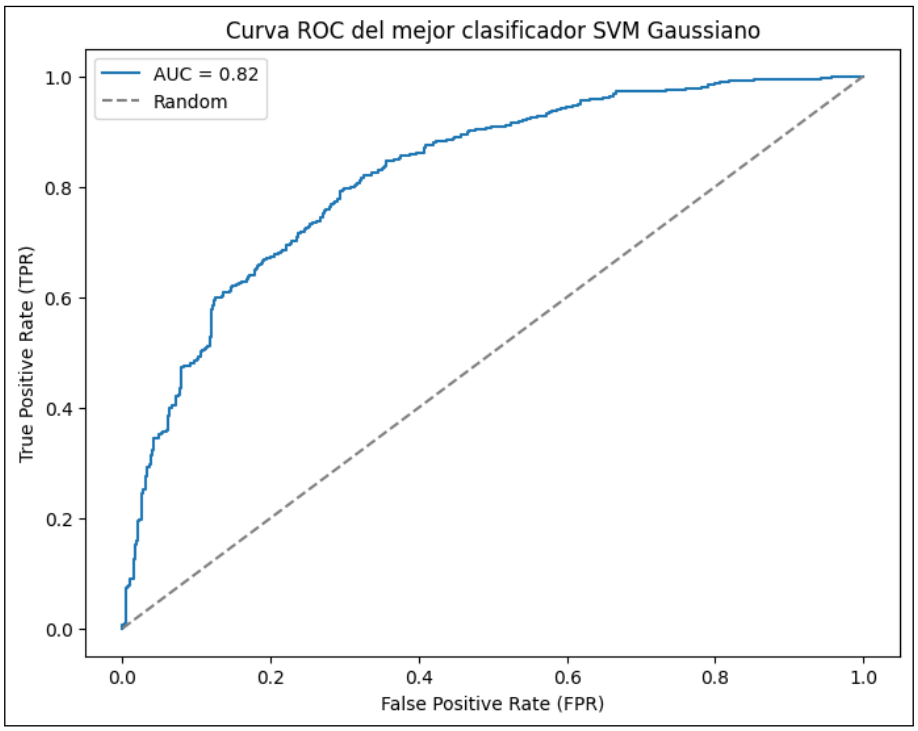


Figura 11: Curva ROC y valor AUC del mejor clasificador SVM gaussiano.

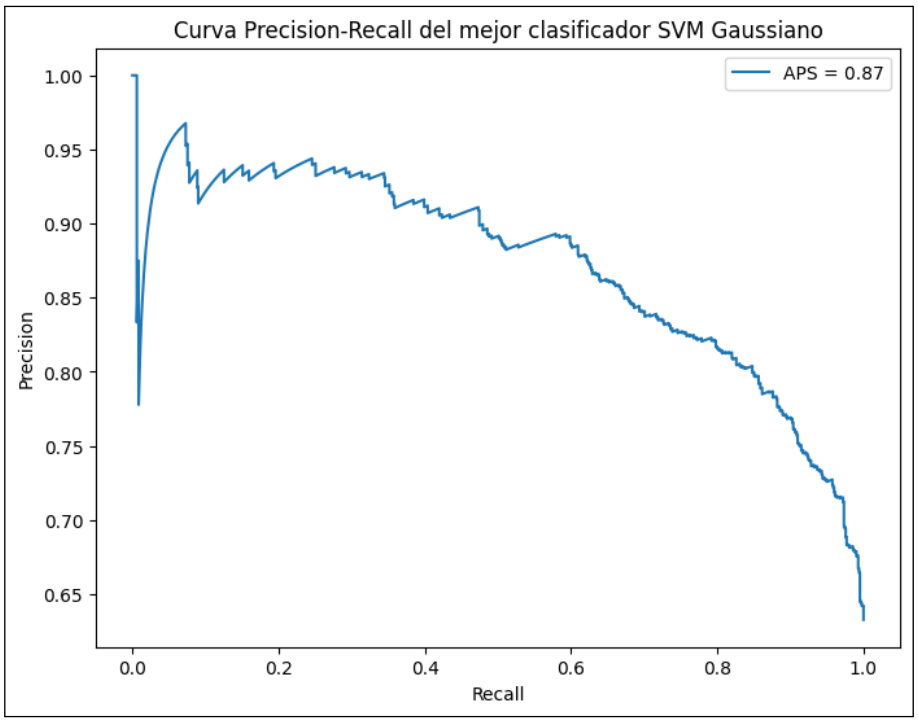


Figura 12: Curva Precision-Recall y valor APS del mejor clasificador SVM gaussiano.

De la **matriz de confusión** representada en 9 y 10, es posible observar que el valor de TP es superior al resto, incluso siendo mayor a la mitad del total de casos, le sigue el valor TN, luego el valor de FN y por ultimo el de FP, por lo que el clasificador obtenido es «muy bueno», ya que los primeros dos valores mencionados son aciertos, y suman un 77 %, mientras que el 23 % restante son los errores.
De la curva **ROC** 11 se puede mencionar que es una curva «buena», pues la pendiente inicial es bastante pronunciada, es una curva suave y se aleja bastante de la curva **Random**. Respecto al valor **AUC** obtenido, se puede mencionar que es alto, lo suficientemente como para considerar que el clasificador es muy bueno.
De la curva **Precision-Recall** 12 se puede mencionar que es una curva «buena», pues la pendiente inicial es pronunciada, pero no es una curva suave. Respecto al valor **APS** obtenido, se puede mencionar que es alto e indica que el modelo tiene una alta precisión al clasificar los elementos de la clase positiva. Esto confirma lo mencionado en el análisis de la **matriz de confusión**.

1.7. Evaluación y comparación de los mejores clasificadores obtenidos

En la presente subsección se compararan los mejores clasificadores obtenidos en 1.4.1, 1.5.1 y 1.6.1.
Para realizar la comparación, se evaluaran a partir de las métricas señaladas en 1.4.2.
Es importante mencionar que en esta ocasión se realizo la predicción sobre el conjunto de pruebas.
Las gráficas generadas se presentan a continuación.

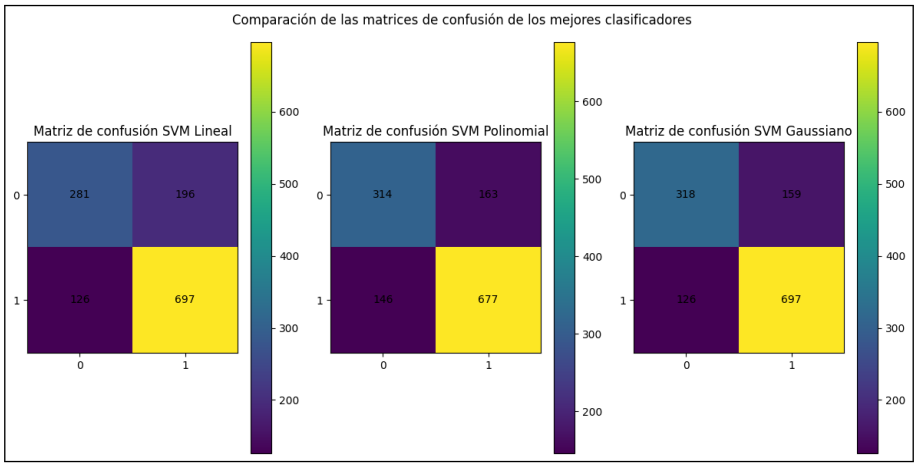


Figura 13: Matrices de confusión de los mejores clasificadores.

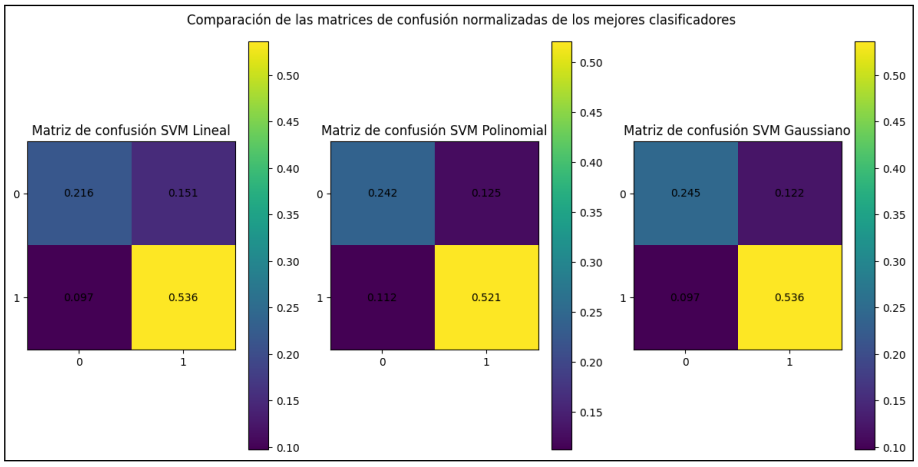


Figura 14: Matrices de confusión normalizadas de los mejores clasificadores.

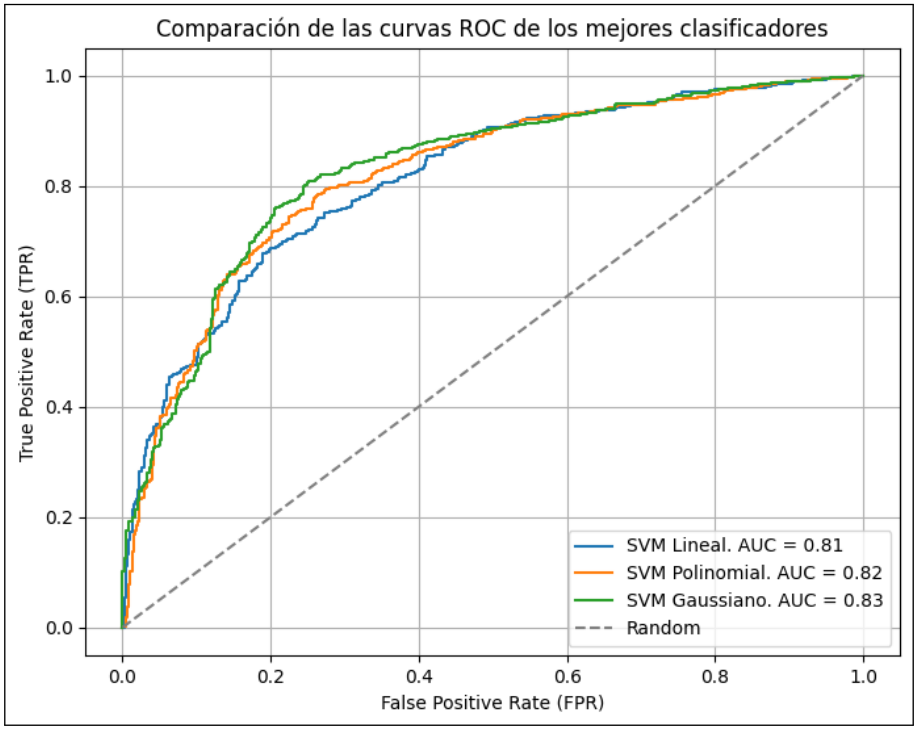


Figura 15: Curva ROC y valor AUC de los mejores clasificadores.

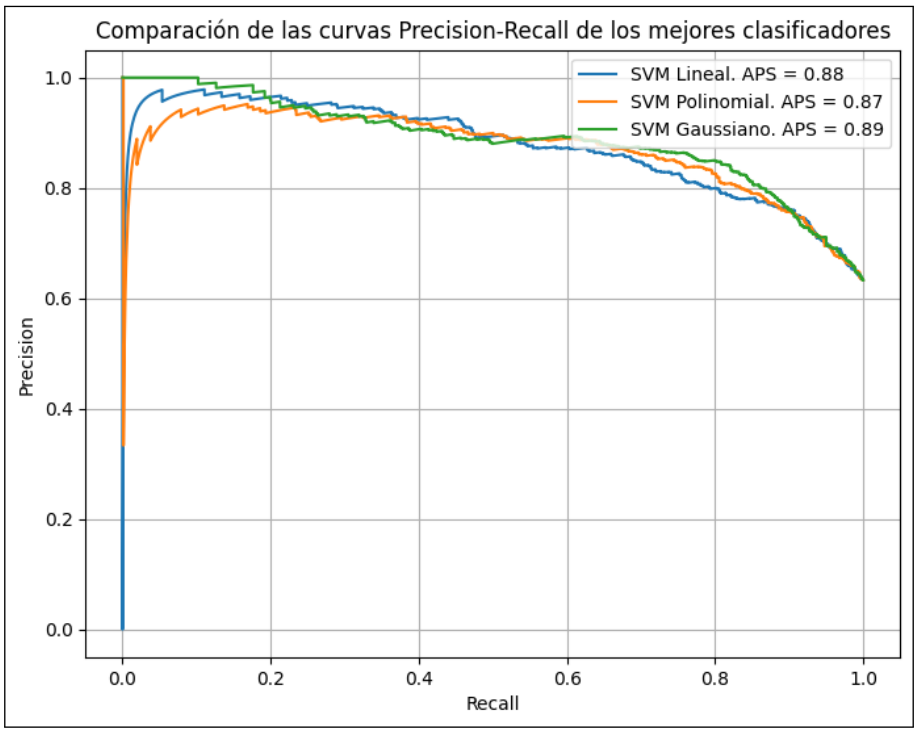


Figura 16: Curva Precision-Recall y valor APS de los mejores clasificadores.

De la **matriz de confusión** representada en 13 y 14, es posible observar que existe una tendencia, la cual correspone a que el valor de TP es superior al resto, incluso siendo mayor a la mitad del total de casos, le sigue el valor TN, luego el valor de FN y por ultimo el de FP, por lo que todos los clasificadores obtenidos son «buenos», ya que los primeros dos valores mencionados son aciertos y los que le siguen son los errores.
Además, es posible observar que el mejor clasificador según esta métrica seria el SVM Gaussiano, pues su cantidad de aciertos es mayor que la de los otros clasificadores.
De la curva **ROC** 15 se puede mencionar que son curvas «buenas», pues las pendientes iniciales son bastantes pronunciadas, son curvas suaves y se alejan bastante de la curva **Random**. Respecto a los valores **AUC** obtenidos, se puede mencionar que son altos, lo suficientemente como para considerar que el clasificador es muy bueno.
Además, es posible observar que, nuevamente, el mejor clasificador según esta métrica seria el SVM Gaussiano, pues es la curva que más se aleja de la curva **Random**, lo que se refleja en su mayor valor de **AUC**, el cual es mayor que la de los otros clasificadores.
De la curva **Precision-Recall** 16 se puede mencionar que son curvas «relativamente buenas», pues las pendientes iniciales son pronunciadas, pero no son curvas suaves, sobre todo a medida que se acercan al 0 en el eje **Recall**. Respecto a los valores **APS** obtenidos, se puede mencionar que son bastante altos e indican que los modelos tiene una alta precisión al clasificar los elementos de la clase positiva. Esto confirma la tendencia mencionada en el análisis de la **matriz de confusión**.
Además, es posible observar que, nuevamente, el mejor clasificador según esta métrica seria

el SVM Gaussiano, pues es la curva más suave, con la menor caída en el 0 del eje **Recall** y lo que se refleja en su valor de **APS**, el cual es mayor que la de los otros clasificadores.
Es por lo anterior que, de manera tentativa, el mejor clasificador es el Support Vector Machine con kernel Gaussiano.

1.8. Análisis del rendimiento de los mejores clasificadores obtenidos

En la presente subsección se realiza el análisis del rendimiento de los mejores clasificadores entrenados, para lo cual se consideran las métricas obtenidas en 1.7 y el tiempo de entrenamiento.
Para realizar lo anterior se modelo un problema de optimización, el cual se muestra en 1.8.

$$\begin{aligned} \text{maximizar} \quad & \sum_{i=1}^{n}\sum_{j=1}^{m} w_{ij} \cdot m_{ij} \\ \text{sujeto a} \quad & \sum_{j=1}^{m} w_{ij} = 1, \quad \forall i = 1, \ldots, n \\ & w_{ij} \neq 0, \quad \forall i = 1, \ldots, n, \forall j = 1, \ldots, m \end{aligned}$$

Donde:

- n es el número de clasificadores.
- m es el número de métricas por clasificador.
- w_{ij} es el peso asignado a la métrica j del clasificador i.
- m_{ij} es el valor de la métrica j del clasificador i.

Para modelar dicho problema en python, el primer paso es obtener las métricas a utilizar. El código utilizado para la obtención de las métricas se adjunta en 7.

Código 7: Obtención de las métricas para el problema de optimización.

```
TrainingTimes = [FinalTrainingTimeLinear - InitialTrainingTimeLinear,
    ↪ FinalTrainingTimePolynomial - InitialTrainingTimePolynomial,
    ↪ FinalTrainingTimeRBF - InitialTrainingTimeRBF]
AUCValues = [AUCLinear, AUCPolynomial, AUCRBF]
AveragePrecisionScoreValues = [AveragePrecisionScoreLinear,
    ↪ AveragePrecisionScorePolynomial, AveragePrecisionScoreRBF]
TNLinear, FPLinear, FNLinear, TPLinear = ConfusionMatrixLinear.ravel()
TrueLinear, FalseLinear = TNLinear + TPLinear, FPLinear + FNLinear
TNPolynomial, FPPolynomial, FNPolynomial, TPPolynomial = ConfusionMatrixPolynomial.
    ↪ ravel()
TruePolynomial, FalsePolynomial = TNPolynomial + TPPolynomial, FPPolynomial +
    ↪ FNPolynomial
```

```
TNRBF, FPRBF, FNRBF, TPRBF = ConfusionMatrixRBF.ravel()
TrueRBF, FalseRBF = TNRBF + TPRBF, FPRBF + FNRBF
Succes = [TrueLinear, TruePolynomial, TrueRBF]
Errors = [FalseLinear, FalsePolynomial, FalseRBF]
TPRValues = [TPRLinear, TPRPolynomial, TPRRBF]
FPRValues = [FPRLinear, FPRPolynomial, FPRRBF]
PrecisionValues = [PrecisionLinear, PrecisionPolynomial, PrecisionRBF]
RecallValues = [RecallLinear, RecallPolynomial, RecallRBF]
def average_metrics(Values):
  AverageValues = []
  for L in Values:
    Average = sum(L) / len(L)
    AverageValues.append(Average)
  return AverageValues
AverageTPRValues = average_metrics(TPRValues)
AveragePrecisionValues = average_metrics(PrecisionValues)
```

El segundo paso para el modelado del problema es definir las métricas que serán utilizadas. Como en el enunciado se nos indica utilizar las métricas obtenidas en 1.7 y el tiempo de entrenamiento, las métricas utilizadas serán:

1. Tiempo de entrenamiento (`TrainingTimes`).
2. Valores de AUC (`AUCValues`).
3. Valores de APS (`AveragePrecisionScoreValues`).
4. Aciertos (`Succes`).
5. Errores (`Errors`).
6. Promedio de los valores de TPR (`AverageTPRValues`).
7. Promedio de los valores de Precision (`AvergaPrecisionValues`).

El tercer paso para el modelado es definir cuales métricas se desean minimizar y cuales maximizar. En este caso, se buscara minimizar el tiempo de entrenamiento y los errores, dando especial énfasis en el tiempo de entrenamiento. El resto de métricas se buscara maximizar.
El cuarto paso para el modelado es definir los pesos de cada métrica. Aquellas que se busquen minimizar deben tener valores negativos. Los valores, de manera respectiva, serán:

1. -0.4.
2. 0.3.
3. 0.3.
4. 0.3.

5. -0.1.

6. 0.3.

7. 0.3.

El ultimo paso es plasmar el problema en python. El código utilizado para modelar el problema de optimización en python se adjunta en 8.

Código 8: Problema de optimización en python.

```
Metrics = [TrainingTimes, AUCValues, AveragePrecisionScoreValues, Succes, Errors,
    ↪ AverageTPRValues, AveragePrecisionValues]
Weights = [-0.4, 0.3, 0.3, 0.3, -0.1, 0.3, 0.3]
WeightedSums = []
for i in range(len(Metrics[0])):
  IndexSum = sum(Metric[i] * Weight for Metric, Weight in zip(Metrics, Weights[:len(Metrics
    ↪ )]))
  WeightedSums.append(IndexSum)
MaxSum = max(WeightedSums)
MaxIndex = WeightedSums.index(MaxSum)
```

El resultado obtenido de la ejecución de 8 se adjunta en 6.

Tabla 6: Resultados problema de optimización.

Sumatorias ponderadas.	[255.55445993278624, 221.39227455601682, 267.9111340864503]
Máximo de las sumatorias ponderadas.	267.9111340864503
Clasificador optimo.	Support Vector Machine Gaussiano.

Es de suma importancia recalcar que el resultado obtenido en 6 es, en parte, debido a como fue modelado el problema de optimización, sobretodo los valores de los pesos utilizados. Aun así, el resultado concuerda con el obtenido en el análisis y comparación de los clasificadores realizado en 1.7, es decir, ambos señalan como mejor clasificador al Support Vector Machine con kernel Gaussiano. En este caso, al mejor clasificador le sigue el Support Vector Machine con kernel Lineal, quedando en ultimo lugar el Support Vector Machine con kernel Polinomial, seguramente debido a su alto tiempo de entrenamiento.
Es por eso que uno debe decidir a que métrica darle más importancia, pues en la teoría, el SVM con kernel Polinomial es superior al SVM con kernel Lineal. Esto también se puede apreciar en 1.4.2 y 1.5.2, en donde se observan mejores métricas para el SVM con kernel polinomial, incluso si se observa 1.6.2, las métricas obtenidas para el SVM con kernel polinomial son también superiores a las métricas obtenidas para el SVM con kernel gaussiano.

2. Conclusiones

En la presente sección se presentaran las conclusiones generales del informe.

2.1. Aprendizajes

A continuación, se describen mis aprendizajes.

Clasificador Support Vector Machine He aprendido como implementar, mediante el uso de la librería **scikit-learn**, un tipo de clasificador SVM, el **SVC** (Support Vector Classifier), el cual posee como hiper-parámetro fundamental **C**, hiper-parámetro que controla la regularización, es decir, es el hiper-parámetro que controla la compensación entre tener un límite de decisión suave y clasificar correctamente la mayoría de las muestras del conjunto de entrenamiento. Por lo anterior, creo que si en un futuro debiese implementar nuevamente un SVM como clasificador, sea SVC o algún otro, seria más sencillo de implementar.

He aprendido que los SVM son principalmente utilizados en problemas de clasificación, pero que también pueden ser utilizados en problemas de regresión y detección de outliers.

He aprendido que el SVM es una buena alternativa en casos en los que el deep learning no es aplicable debido al tamaño del conjunto de datos/entrenamiento o bajas capacidades computacionales.

Kernel He aprendido la gran importancia de la correcta elección del kernel a utilizar en un SVM, pues la correcta elección del mismo permite la correcta separación de las muestras del conjunto de entrenamiento. He aprendido que si bien como «regla general» se elige primero los kernels Gaussianos o Polinomiales, puede ocurrir que estos demoren más tiempo en entrenarse, por lo cual es importante evaluar dicho factor en el caso de la elección del kernel. Aun así, parece ser que los resultados obtenidos al realizar una predicción sobre los conjuntos de validación y prueba son superiores a los obtenidos con un kernel Lineal, lo cual parece respaldar la «regla general».

Hiper-parámetro C He aprendido que el valor de este hiper-parámetro está estrechamente relacionado con el conjunto de datos de entrenamiento utilizados, pues, como mencione previamente, dicho hiper-parámetro es el encargado de la regularización, lo que lleva a un modelo que se ajuste mejor a los datos de entrenamiento y si el valor es muy alto puede ocurrir un sobre ajuste en el caso de un modelo muy complejo. También puede ocurrir que para un menor valor de **C**, el modelo se vuelva muy tolerante al ruido y a los outliers del conjunto de entrenamiento. Entonces, la elección adecuada de **C** influye en la capacidad del SVC para generalizar, es decir, influye en la capacidad de predecir correctamente datos nuevos (no vistos en el entrenamiento). He aprendido, que otros factores que influyen en la elección de **C** son el problema de clasificación y la partición de validación cruzada. Todo lo anterior podría explicar porque el valor de **C** obtenido en los tres clasificadores implementados es el mismo.

Búsqueda de la grilla He aprendido que la búsqueda de la grilla es una técnica que se utiliza para encontrar los mejores hiper-parámetros de un modelo, logrando así la automatización del proceso de selección de hiper-parámetros. He aprendido acerca de su funcionamiento, de los pasos que realiza y de como el conjunto de validación es sumamente relevante para la elección final de los hiper-parámetros, pues aquellos hiper-parámetros que posean el mejor rendimiento en el conjunto de validación serán los elegidos. He aprendido a implementar dicha técnica mediante el uso de la librería *scikit-learn*.

Tiempos de entrenamiento He aprendido acerca de la relevancia de los tiempos de entrenamiento a la hora de elegir un modelo. Si se requiere un tiempo de respuesta rápido, como seria realizar predicciones en tiempo real, el tiempo de entrenamiento es de suma relevancia, pues si el modelo posee un alto tiempo de entrenamiento, podría ocurrir que las respuestas sean lentas. He aprendido que el tiempo de entrenamiento tiene relación directa con la complejidad del modelo utilizado y con la cantidad de datos de entrenamiento. Mientras más simple sea el modelo, menor sera su tiempo de entrenamiento. Mientras mayor sea la cantidad de datos, usualmente aumentara el tiempo de entrenamiento y en caso de que se utilicen algoritmos que dependen de la cantidad de datos, el tiempo de entrenamiento aumentara con certeza.

Métricas de evaluación He aprendido acerca de la relevancia de las métricas a la hora de comparar distintos clasificadores. En particular, de las nuevas métricas he aprendido que:

AUC Es aquel valor que representa el área bajo la curva ROC y representa una medida de la capacidad de discriminación del modelo, pues mientras más cercano sea el valor a 1, mejor capacidad de discriminación tendrá un modelo, en cambio un valor cercano a 0.5 indica un rendimiento aleatorio.

APS El APS es el area bajo la curva Precision-Recall y es una métrica que representa la precisión de un modelo en un problema de clasificación binaria, pues esta métrica considera todos los umbrales posibles del modelo.

Análisis del rendimiento de los clasificadores He aprendido acerca de la relevancia de las métricas para el análisis del rendimiento de un modelo clasificador pero, he aprendido que el tiempo de entrenamiento también es de suma relevancia.
He aprendido que una buena forma para realizar un análisis del rendimiento de varios clasificadores es modelar un problema de optimización que permita comparar de manera integral las diversas métricas que existen.

2.2. Dificultades

- Desconocimiento acerca de la validación cruzada.
- Desconocimiento acerca de la búsqueda de la grilla.
- Comprensión y aplicación de la función *PredefinedSplit*.

- Aplicación de la función *GridSearchCV*.

2.3. Evaluación de resultados

Los resultados obtenidos son los que se esperan en función de la teoría, pues es claro que el mejor clasificador según las métricas es el Support Vector Machine con kernel Gaussiano, seguido por el Support Vector Machine con kernel Polinomial y por ultimo el Support Vector Machine con kernel Lineal, por lo que se comprueba la regla de utilizar primero los kernels Gaussiano y Polinomial.
Además, se comprueba que la frontera de decisión obtenida por un modelo varia según el kernel utilizado, pues como se pudo ver, los valores de las métricas obtenidas son diferentes para cada modelo entrenado.

Anexo A. Código

Código A.1: Jupyter Notebook.

```
# -*- coding: utf-8 -*-
"""Tarea 2 – Clasificador Support Vector Machine EL4106 - Diego Vega.ipynb

Automatically generated by Colab.

Original file is located at
   https://colab.research.google.com/drive/1QDTvq1SJaptFHtT7l82Y5-1EIKStxwuC

# **Importar librerias**
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import sklearn
import time
from sklearn.model_selection import train_test_split, PredefinedSplit, GridSearchCV
from sklearn.preprocessing import MinMaxScaler
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, roc_curve, auc,
    ↪ precision_recall_curve, average_precision_score

"""# **Desarrollo**

## **Manejo de datos**

### Lectura del Data Set y modifcación de columnas
```

```
"""

DataSet = 'winequality-all.csv'
DataFrame = pd.read_csv(DataSet, sep=';')
print('Columnas del DataFrame de winequality-all.csv: ')
print(DataFrame.columns)
print('\n')
def high_or_low(quality):
  if quality >= 6:
    return 1
  else:
    return 0
DataFrame['class'] = DataFrame['quality'].apply(high_or_low)
print('Columnas del DataFrame de winequality-all.csv con columna class: ')
print(DataFrame.columns)
print('\n')

DataFrame = DataFrame.drop('quality', axis=1)
print('Columnas del DataFrame de winequality-all.csv con columna class y sin quality: ')
print(DataFrame.columns)

"""### Dividir Data Set en conjuntos de entrenamiento, validación y prueba"""

X = DataFrame.drop('class', axis=1)
y = DataFrame['class']

TrainRatio = 0.6
TestRatio = 0.2
ValidationRatio = 0.2

XTrain, XTemporary, yTrain, yTemporary = train_test_split(X, y, test_size=1-TrainRatio,
    ↪ random_state=3, stratify=y)
XValidation, XTest, yValidation, yTest = train_test_split(XTemporary, yTemporary,
    ↪ test_size=0.5, random_state=3, stratify=yTemporary)

OriginalProportion = [round((y==0).sum()/len(y), 4), round((y==1).sum()/len(y), 4)]
TrainProportion = [round((yTrain==0).sum()/len(yTrain), 4), round((yTrain==1).sum()/
    ↪ len(yTrain), 4)]
ValidationProportion = [round((yValidation==0).sum()/len(yValidation), 4), round((
    ↪ yValidation==1).sum()/len(yValidation), 4)]
TestProportion = [round((yTest==0).sum()/len(yTest), 4), round((yTest==1).sum()/len(
    ↪ yTest), 4)]

print(f'Proporción de las clases en el DataFrame original: {OriginalProportion}')
print(f'Proporción de las clases en el DataFrame de entrenamiento: {TrainProportion}')
print(f'Proporción de las clases en el DataFrame de validación: {ValidationProportion}')
print(f'Proporción de las clases en el DataFrame de prueba: {TestProportion}')
```

```
69
70 """### Normalizar las características de todos los conjuntos"""
71
72 Scaler = MinMaxScaler()
73 Scaler.fit(XTrain)
74
75 XTrainNormalized = Scaler.transform(XTrain)
76
77 XValidationNormalized = Scaler.transform(XValidation)
78
79 XTestNormalized = Scaler.transform(XTest)
80
81 """## **Support Vector Machine Lineal**
82
83 ### Entrenamiento del clasificador
84 """
85
86 Indexs = np.zeros(len(XTrainNormalized) + len(XValidationNormalized))
87 Indexs[:len(XTrainNormalized)] = -1
88 PreDefinedSplit = PredefinedSplit(test_fold=Indexs)
89 ParametersGrid = {'C': [0.01, 0.1, 1, 10, 100]}
90 SVML = SVC(kernel='linear')
91 GridSearch = GridSearchCV(estimator=SVML, param_grid=ParametersGrid, cv=
       ↪ PreDefinedSplit)
92 InitialTrainingTimeLinear = time.time()
93 GridSearch.fit(np.vstack((XTrainNormalized, XValidationNormalized)), np.hstack((yTrain,
       ↪ yValidation)))
94 FinalTrainingTimeLinear = time.time()
95 BestSVML = GridSearch.best_estimator_
96 BestSVML.fit(XTrainNormalized, yTrain)
97 print(f'El tiempo de entrenamiento es: {FinalTrainingTimeLinear - InitialTrainingTimeLinear
       ↪ }')
98 print(f'El mejor clasificador SVM Lineal es: SVC(C={BestSVML.C}, kernel={BestSVML.
       ↪ kernel})')
99
100 """### Evaluación del mejor clasificador"""
101
102 yPredictedValidation = BestSVML.predict(XValidationNormalized)
103 Labels=[0, 1]
104
105 ConfusionMatrix = confusion_matrix(yValidation, yPredictedValidation, labels=Labels)
106 Display = ConfusionMatrixDisplay(confusion_matrix=ConfusionMatrix, display_labels=
       ↪ Labels)
107 Display.plot()
108 plt.title('Matriz de Confusión del mejor clasificador SVM Lineal')
109 plt.show()
110
```

```
111 print('\n')
112
113 ConfusionMatrixNormalized = confusion_matrix(yValidation, yPredictedValidation, labels=
        ↪ Labels, normalize='all')
114 TNNormalized, FPNormalized, FNNormalized, TPNormalized = ConfusionMatrixNormalized
        ↪ .ravel()
115 DisplayNormalized = ConfusionMatrixDisplay(confusion_matrix=
        ↪ ConfusionMatrixNormalized, display_labels=Labels)
116 DisplayNormalized.plot()
117 plt.title('Matriz de Confusión Normalizada del mejor clasificador SVM Lineal')
118 plt.show()
119
120 print('\n')
121
122 DecisionFunction = BestSVML.decision_function(XValidationNormalized)
123
124 FPR, TPR, Thresholds = roc_curve(yValidation, DecisionFunction)
125 AUC = auc(FPR, TPR)
126 plt.figure(figsize=(8, 6))
127 plt.plot(FPR, TPR, label=f'AUC = {AUC:.2f}')
128 plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')
129 plt.xlabel('False Positive Rate (FPR)')
130 plt.ylabel('True Positive Rate (TPR)')
131 plt.title('Curva ROC del mejor clasificador SVM Lineal')
132 plt.legend()
133 plt.show()
134
135 print('\n')
136
137 Precision, Recall, Threshold = precision_recall_curve(yValidation, DecisionFunction)
138 AveragePrecisionScore = average_precision_score(yValidation, DecisionFunction)
139 plt.figure(figsize=(8, 6))
140 plt.plot(Recall, Precision, label=f'APS = {AveragePrecisionScore:.2f}')
141 plt.xlabel('Recall')
142 plt.ylabel('Precision')
143 plt.title('Curva Precision-Recall del mejor clasificador SVM Lineal')
144 plt.legend()
145 plt.show()
146
147 """## **Support Vector Machine Polinomial**
148
149 ### Entrenamiento del clasificador
150 """
151
152 ParametersGrid = {'C': [0.1, 1, 10], 'degree': [2, 3, 4]}
153 SVMP = SVC(kernel='poly', coef0=1)
```

```
GridSearch = GridSearchCV(estimator=SVMP, param_grid=ParametersGrid, cv=PreDefinedSplit)
InitialTrainingTimePolynomial = time.time()
GridSearch.fit(np.vstack((XTrainNormalized, XValidationNormalized)), np.hstack((yTrain, yValidation)))
FinalTrainingTimePolynomial = time.time()
BestSVMP = GridSearch.best_estimator_
BestSVMP.fit(XTrainNormalized, yTrain)
print(f'El tiempo de entrenamiento es: {FinalTrainingTimePolynomial - InitialTrainingTimePolynomial}')
print(f'El mejor clasificador SVM Polinomial es: SVC(C={BestSVMP.C}, degree={BestSVMP.degree}, coef0={BestSVMP.coef0}, kernel={BestSVMP.kernel})')

"""### Evaluación del mejor clasificador"""

yPredictedValidation = BestSVMP.predict(XValidationNormalized)
Labels=[0, 1]

ConfusionMatrix = confusion_matrix(yValidation, yPredictedValidation, labels=Labels)
Display = ConfusionMatrixDisplay(confusion_matrix=ConfusionMatrix, display_labels=Labels)
Display.plot()
plt.title('Matriz de Confusión del mejor clasificador SVM Polinomial')
plt.show()

print('\n')

ConfusionMatrixNormalized = confusion_matrix(yValidation, yPredictedValidation, labels=Labels, normalize='all')
TNNormalized, FPNormalized, FNNormalized, TPNormalized = ConfusionMatrixNormalized.ravel()
DisplayNormalized = ConfusionMatrixDisplay(confusion_matrix=ConfusionMatrixNormalized, display_labels=Labels)
DisplayNormalized.plot()
plt.title('Matriz de Confusión Normalizada del mejor clasificador SVM Polinomial')
plt.show()

print('\n')

DecisionFunction = BestSVMP.decision_function(XValidationNormalized)

FPR, TPR, Thresholds = roc_curve(yValidation, DecisionFunction)
AUC = auc(FPR, TPR)
plt.figure(figsize=(8, 6))
plt.plot(FPR, TPR, label=f'AUC = {AUC:.2f}')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')
plt.xlabel('False Positive Rate (FPR)')
```

```
plt.ylabel('True Positive Rate (TPR)')
plt.title('Curva ROC del mejor clasificador SVM Polinomial')
plt.legend()
plt.show()

print('\n')

Precision, Recall, Threshold = precision_recall_curve(yValidation, DecisionFunction)
AveragePrecisionScore = average_precision_score(yValidation, DecisionFunction)
plt.figure(figsize=(8, 6))
plt.plot(Recall, Precision, label=f'APS = {AveragePrecisionScore:.2f}')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Curva Precision-Recall del mejor clasificador SVM Polinomial')
plt.legend()
plt.show()

"""## **Support Vector Machine RBF**

### Entrenamiento del clasificador
"""

ParametersGrid = {'C': [0.01, 0.1, 1, 10, 100], 'gamma': [0.01, 0.1, 1, 10]}
SVMRBF = SVC(kernel='rbf')
GridSearch = GridSearchCV(estimator=SVMRBF, param_grid=ParametersGrid, cv=
    ↪ PreDefinedSplit)
InitialTrainingTimeRBF = time.time()
GridSearch.fit(np.vstack((XTrainNormalized, XValidationNormalized)), np.hstack((yTrain,
    ↪ yValidation)))
FinalTrainingTimeRBF = time.time()
BestSVMRBF = GridSearch.best_estimator_
BestSVMRBF.fit(XTrainNormalized, yTrain)
print(f'El tiempo de entrenamiento es: {FinalTrainingTimeRBF - InitialTrainingTimeRBF}')
print(f'El mejor clasificador SVM Gaussiano es: SVC(C={BestSVMRBF.C}, gamma={
    ↪ BestSVMRBF.gamma}, kernel={BestSVMRBF.kernel})')

"""### Evaluación del mejor clasificador"""

yPredictedValidation = BestSVMRBF.predict(XValidationNormalized)
Labels=[0, 1]

ConfusionMatrix = confusion_matrix(yValidation, yPredictedValidation, labels=Labels)
Display = ConfusionMatrixDisplay(confusion_matrix=ConfusionMatrix, display_labels=
    ↪ Labels)
Display.plot()
plt.title('Matriz de Confusión del mejor clasificador SVM Gaussiano')
plt.show()
```

```
236
237 print('\n')
238
239 ConfusionMatrixNormalized = confusion_matrix(yValidation, yPredictedValidation, labels=
    ↪ Labels, normalize='all')
240 TNNormalized, FPNormalized, FNNormalized, TPNormalized = ConfusionMatrixNormalized
    ↪ .ravel()
241 DisplayNormalized = ConfusionMatrixDisplay(confusion_matrix=
    ↪ ConfusionMatrixNormalized, display_labels=Labels)
242 DisplayNormalized.plot()
243 plt.title('Matriz de Confusión Normalizada del mejor clasificador SVM Gaussiano')
244 plt.show()
245
246 print('\n')
247
248 DecisionFunction = BestSVMRBF.decision_function(XValidationNormalized)
249
250 FPR, TPR, Thresholds = roc_curve(yValidation, DecisionFunction)
251 AUC = auc(FPR, TPR)
252 plt.figure(figsize=(8, 6))
253 plt.plot(FPR, TPR, label=f'AUC = {AUC:.2f}')
254 plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')
255 plt.xlabel('False Positive Rate (FPR)')
256 plt.ylabel('True Positive Rate (TPR)')
257 plt.title('Curva ROC del mejor clasificador SVM Gaussiano')
258 plt.legend()
259 plt.show()
260
261 print('\n')
262
263 Precision, Recall, Threshold = precision_recall_curve(yValidation, DecisionFunction)
264 AveragePrecisionScore = average_precision_score(yValidation, DecisionFunction)
265 plt.figure(figsize=(8, 6))
266 plt.plot(Recall, Precision, label=f'APS = {AveragePrecisionScore:.2f}')
267 plt.xlabel('Recall')
268 plt.ylabel('Precision')
269 plt.title('Curva Precision-Recall del mejor clasificador SVM Gaussiano')
270 plt.legend()
271 plt.show()
272
273 """## **Evaluación y comparación de los mejores clasificadores obtenidos**"""
274
275 yPredictedLinear, yPredictedPolynomial, yPredictedRBF = BestSVML.predict(
    ↪ XTestNormalized), BestSVMP.predict(XTestNormalized), BestSVMRBF.predict(
    ↪ XTestNormalized)
276 Labels = [0, 1]
277
```

```
278 ConfusionMatrixLinear = confusion_matrix(yTest, yPredictedLinear, labels=Labels)
279 ConfusionMatrixPolynomial = confusion_matrix(yTest, yPredictedPolynomial, labels=
        ↪ Labels)
280 ConfusionMatrixRBF = confusion_matrix(yTest, yPredictedRBF, labels=Labels)
281
282 fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(12, 6))
283 fig.suptitle('Comparación de las matrices de confusión de los mejores clasificadores')
284 ILinear = ax1.imshow(ConfusionMatrixLinear, cmap=plt.cm.viridis)
285 ax1.set_xticks(np.arange(len(Labels)), labels=Labels)
286 ax1.set_yticks(np.arange(len(Labels)), labels=Labels)
287 for i in range(len(Labels)):
288   for j in range(len(Labels)):
289     text = ax1.text(j, i, f'{ConfusionMatrixLinear[i, j]}', ha='center', va='center', color='
        ↪ black')
290 ax1.set_title('Matriz de confusión SVM Lineal')
291 fig.colorbar(ILinear, ax=ax1)
292 IPolynomial = ax2.imshow(ConfusionMatrixPolynomial, cmap=plt.cm.viridis)
293 ax2.set_xticks(np.arange(len(Labels)), labels=Labels)
294 ax2.set_yticks(np.arange(len(Labels)), labels=Labels)
295 for i in range(len(Labels)):
296   for j in range(len(Labels)):
297     text = ax2.text(j, i, f'{ConfusionMatrixPolynomial[i, j]}', ha='center', va='center', color=
        ↪ 'black')
298 ax2.set_title('Matriz de confusión SVM Polinomial')
299 fig.colorbar(IPolynomial, ax=ax2)
300 IRBF = ax3.imshow(ConfusionMatrixRBF, cmap=plt.cm.viridis)
301 ax3.set_xticks(np.arange(len(Labels)), labels=Labels)
302 ax3.set_yticks(np.arange(len(Labels)), labels=Labels)
303 for i in range(len(Labels)):
304   for j in range(len(Labels)):
305     text = ax3.text(j, i, f'{ConfusionMatrixRBF[i, j]}', ha='center', va='center', color='black'
        ↪ )
306 ax3.set_title('Matriz de confusión SVM Gaussiano')
307 fig.colorbar(IRBF, ax=ax3)
308 plt.tight_layout()
309 plt.show()
310
311 print('\n')
312
313 ConfusionMatrixLinearNormalized = confusion_matrix(yTest, yPredictedLinear, labels=
        ↪ Labels, normalize='all')
314 ConfusionMatrixPolynomialNormalized = confusion_matrix(yTest, yPredictedPolynomial,
        ↪ labels=Labels, normalize='all')
315 ConfusionMatrixRBFNormalized = confusion_matrix(yTest, yPredictedRBF, labels=Labels,
        ↪  normalize='all')
316
317 fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(12, 6))
```

```
318 fig.suptitle('Comparación de las matrices de confusión normalizadas de los mejores
        ↪ clasificadores')
319 ILinearNormalized = ax1.imshow(ConfusionMatrixLinearNormalized, cmap=plt.cm.viridis)
320 ax1.set_xticks(np.arange(len(Labels)), labels=Labels)
321 ax1.set_yticks(np.arange(len(Labels)), labels=Labels)
322 for i in range(len(Labels)):
323   for j in range(len(Labels)):
324     text = ax1.text(j, i, f'{ConfusionMatrixLinearNormalized[i, j]:.3f}', ha='center', va='
        ↪ center', color='black')
325 ax1.set_title('Matriz de confusión SVM Lineal')
326 fig.colorbar(ILinearNormalized, ax=ax1)
327 IPolynomialNormalized = ax2.imshow(ConfusionMatrixPolynomialNormalized, cmap=plt.cm
        ↪ .viridis)
328 ax2.set_xticks(np.arange(len(Labels)), labels=Labels)
329 ax2.set_yticks(np.arange(len(Labels)), labels=Labels)
330 for i in range(len(Labels)):
331   for j in range(len(Labels)):
332     text = ax2.text(j, i, f'{ConfusionMatrixPolynomialNormalized[i, j]:.3f}', ha='center', va='
        ↪ center', color='black')
333 ax2.set_title('Matriz de confusión SVM Polinomial')
334 fig.colorbar(IPolynomialNormalized, ax=ax2)
335 IRBFNormalized = ax3.imshow(ConfusionMatrixRBFNormalized, cmap=plt.cm.viridis)
336 ax3.set_xticks(np.arange(len(Labels)), labels=Labels)
337 ax3.set_yticks(np.arange(len(Labels)), labels=Labels)
338 for i in range(len(Labels)):
339   for j in range(len(Labels)):
340     text = ax3.text(j, i, f'{ConfusionMatrixRBFNormalized[i, j]:.3f}', ha='center', va='center'
        ↪ , color='black')
341 ax3.set_title('Matriz de confusión SVM Gaussiano')
342 fig.colorbar(IRBFNormalized, ax=ax3)
343 plt.tight_layout()
344 plt.show()
345
346 print('\n')
347
348 DecisionFunctionLinear, DecisionFunctionPolynomial, DecisionFunctionRBF = BestSVML.
        ↪ decision_function(XTestNormalized), \
349                                                     BestSVMP.decision_function(
        ↪ XTestNormalized), \
350                                                     BestSVMRBF.decision_function(
        ↪ XTestNormalized)
351 FPRLinear, TPRLinear, ThresholdsLinear = roc_curve(yTest, DecisionFunctionLinear)
352 AUCLinear = auc(FPRLinear, TPRLinear)
353 FPRPolynomial, TPRPolynomial, ThresholdsPolynomial = roc_curve(yTest,
        ↪ DecisionFunctionPolynomial)
354 AUCPolynomial = auc(FPRPolynomial, TPRPolynomial)
355 FPRRBF, TPRRBF, ThresholdsRBF = roc_curve(yTest, DecisionFunctionRBF)
```

```
AUCRBF = auc(FPRRBF, TPRRBF)
plt.figure(figsize=(8, 6))
plt.plot(FPRLinear, TPRLinear, label=f'SVM Lineal. AUC = {AUCLinear:.2f}')
plt.plot(FPRPolynomial, TPRPolynomial, label=f'SVM Polinomial. AUC = {AUCPolynomial:.2f}')
plt.plot(FPRRBF, TPRRBF, label=f'SVM Gaussiano. AUC = {AUCRBF:.2f}')
plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Random')
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR)')
plt.title('Comparación de las curvas ROC de los mejores clasificadores')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()

print('\n')

PrecisionLinear, RecallLinear, ThresholdLinear = precision_recall_curve(yTest, DecisionFunctionLinear)
AveragePrecisionScoreLinear = average_precision_score(yTest, DecisionFunctionLinear)
PrecisionPolynomial, RecallPolynomial, ThresholdPolynomial = precision_recall_curve(yTest, DecisionFunctionPolynomial)
AveragePrecisionScorePolynomial = average_precision_score(yTest, DecisionFunctionPolynomial)
PrecisionRBF, RecallRBF, ThresholdRBF = precision_recall_curve(yTest, DecisionFunctionRBF)
AveragePrecisionScoreRBF = average_precision_score(yTest, DecisionFunctionRBF)
plt.figure(figsize=(8, 6))
plt.plot(RecallLinear, PrecisionLinear, label=f'SVM Lineal. APS = {AveragePrecisionScoreLinear:.2f}')
plt.plot(RecallPolynomial, PrecisionPolynomial, label=f'SVM Polinomial. APS = {AveragePrecisionScorePolynomial:.2f}')
plt.plot(RecallRBF, PrecisionRBF, label=f'SVM Gaussiano. APS = {AveragePrecisionScoreRBF:.2f}')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Comparación de las curvas Precision-Recall de los mejores clasificadores')
plt.legend(loc='upper right')
plt.grid(True)
plt.show()

"""## **Análisis de rendimiento de los mejores clasificadores obtenidos**"""

TrainingTimes = [FinalTrainingTimeLinear - InitialTrainingTimeLinear, FinalTrainingTimePolynomial - InitialTrainingTimePolynomial, FinalTrainingTimeRBF - InitialTrainingTimeRBF]
AUCValues = [AUCLinear, AUCPolynomial, AUCRBF]
```

```
AveragePrecisionScoreValues = [AveragePrecisionScoreLinear,
    ↪ AveragePrecisionScorePolynomial, AveragePrecisionScoreRBF]
TNLinear, FPLinear, FNLinear, TPLinear = ConfusionMatrixLinear.ravel()
TrueLinear, FalseLinear = TNLinear + TPLinear, FPLinear + FNLinear
TNPolynomial, FPPolynomial, FNPolynomial, TPPolynomial = ConfusionMatrixPolynomial.
    ↪ ravel()
TruePolynomial, FalsePolynomial = TNPolynomial + TPPolynomial, FPPolynomial +
    ↪ FNPolynomial
TNRBF, FPRBF, FNRBF, TPRBF = ConfusionMatrixRBF.ravel()
TrueRBF, FalseRBF = TNRBF + TPRBF, FPRBF + FNRBF
Succes = [TrueLinear, TruePolynomial, TrueRBF]
Errors = [FalseLinear, FalsePolynomial, FalseRBF]
TPRValues = [TPRLinear, TPRPolynomial, TPRRBF]
FPRValues = [FPRLinear, FPRPolynomial, FPRRBF]
PrecisionValues = [PrecisionLinear, PrecisionPolynomial, PrecisionRBF]
RecallValues = [RecallLinear, RecallPolynomial, RecallRBF]

def average_metrics(Values):
  AverageValues = []
  for L in Values:
    Average = sum(L) / len(L)
    AverageValues.append(Average)

  return AverageValues

AverageTPRValues = average_metrics(TPRValues)
AveragePrecisionValues = average_metrics(PrecisionValues)

Metrics = [TrainingTimes, AUCValues, AveragePrecisionScoreValues, Succes, Errors,
    ↪ AverageTPRValues, AveragePrecisionValues]
Weights = [-0.4, 0.3, 0.3, 0.3, -0.1, 0.3, 0.3]
WeightedSums = []
for i in range(len(Metrics[0])):
  IndexSum = sum(Metric[i] * Weight for Metric, Weight in zip(Metrics, Weights[:len(Metrics
    ↪ )]))
  WeightedSums.append(IndexSum)

MaxSum = max(WeightedSums)
MaxIndex = WeightedSums.index(MaxSum)
print(f'Las sumatorias ponderadas son: {WeightedSums}.')
print(f'El máximo de las sumatorias ponderadas es: {MaxSum}.')
if MaxIndex == 0:
  print('El clasificador optimo es el SVM Lineal.')
elif MaxIndex == 1:
  print('El clasificador optimo es el SVM Polinomial.')
elif MaxIndex == 2:
  print('El clasificador optimo es el SVM Gaussiano.')
```